

DAS CHAIN ARCHITECTURE DOSSIER (v1.0)

Autor: Raimund S. Utz **Status:** Strategic Proposal

1. DAS WHITEPAPER: STRATEGIE & VISION

Einleitung

Die **Chain Architecture (CA)** ist eine evolutionäre Weiterentwicklung des Schichtenmodells. Sie ersetzt die radiale Komplexität (Hexagonal) durch **sequenzielle Integrität**. Ein Prozess wird als Kette von Gliedern (**Links**) betrachtet, die durch einen **Context Anchor** gehalten werden.

Kernvorteile

- **Kognitive Entlastung:** Ein linearer Pfad ist für Entwickler sofort lesbar.
 - **Typisierte Sicherheit:** Dank Java Generics passen Glieder nur zusammen, wenn die Schnittstellen stimmen.
 - **Zentralisierte Querschnittsfunktionen:** Logging, Security und Telemetrie leben im Anchor, nicht in der Business-Logik.
-

2. TECHNISCHE REFERENZ: JAVA IMPLEMENTIERUNG

Die Basis-Struktur

```
public interface ChainContext {
    void log(String msg);
    MongoClient getMongoClient();
}

public interface ChainLink<I, O> {
    O process(I input, ChainContext context);
}
```

Beispiel: Order-Processing-Chain

```
@Service
public class OrderService {
    private final ChainContext ctx;

    public void handleOrder(OrderDTO raw) {
        // Die Kette wird im Anchor definiert und ausgeführt
        Chain.start(raw, ctx)
            .link(new ValidationLink()) // Prüft Daten
            .link(new EnrichmentLink()) // Holt Kundendaten
            .link(new MongoPersistence()) // Schreibt in MongoDB
    }
}
```

```
        .execute(raw);
    }
}

// Demo Link:
public class ValidationLink implements ChainLink<Input, Output> {
    public Output process(Input in, ChainContext ctx) {
        if (in.isInvalid()) {
            throw new ChainLinkException("Validation failed",
ctx.getTraceId());
        }
        return new Output(in);
    }
}
```

3. CHEAT SHEET: BEST PRACTICES & ERROR HANDLING

Das "Broken Link" Prinzip

Reißt ein Glied (Exception), stoppt die Kette sofort. Der Anchor fängt den Fehler zentral ab.

- **Fail-Fast:** Validierung immer an den Anfang der Kette setzen.
- **Link-Isolation:** Ein Link darf niemals wissen, was zwei Glieder weiter passiert.
- **Immutabilität:** Daten, die durch die Kette fließen, sollten nach Möglichkeit unveränderlich (Records) sein.

Monitoring & Debugging

Da der **ChainContext** durchgereicht wird, kann jedes Glied automatisch eine **Correlation-ID** an Logs anhängen. Ein Stacktrace in der Chain Architecture ist immer linear und damit trivial zu debuggen.

4. ARGUMENTATIONS-SKRIPT (DEFENSE KIT)

Gegenwind: "Wir brauchen totale Entkopplung (Hexagonal)!"

Antwort: "Totale Entkopplung erzeugt oft 'Leere Abstraktionen'. Wir entkoppeln funktional. Ein **PersistenceLink** kann jederzeit getauscht werden, ohne die Business-Logik der anderen Links zu berühren. Wir sparen uns aber die Mapper-Hölle."

Gegenwind: "Was ist mit dem Domain-Model im Zentrum?"

Antwort: "In der Chain Architecture ist der **Prozess** das Zentrum. Das Domain-Model fließt durch die Kette und wird veredelt. Das ist näher an der Realität von Web-Apps und Microservices als ein statisches Modell in der Mitte eines Hexagons."

Ende des Dossiers